

JS Execution Contexts

06/23/2026 10:55:51



JavaScript Execution Flow (Correct Learning Order)

1. JavaScript is Single Threaded

- JavaScript can execute only **one piece of code at a time**.

2. JavaScript Engine

- The JS Engine (V8, SpiderMonkey, etc.) executes JavaScript code.

3. Memory Heap

- Variables, objects, arrays, and functions are stored in memory heap.

4. Call Stack

- The Call Stack keeps track of which function is currently executing.

5. Execution Context

- An Execution Context is the environment in which JavaScript code runs.

6. Global Execution Context (GEC)

- Whenever a JS file starts, JavaScript creates a Global Execution Context.

7. Memory Creation Phase

- JavaScript first scans the code and allocates memory to variables and functions.

8. Code Execution Phase

- JavaScript then executes code line by line.

9. Hoisting

- Because memory is allocated before execution, variables and functions can be accessed before their declaration.

10. Function Execution Context (FEC)

- Every function call creates its own Execution Context.

11. Call Stack Working

- Function contexts are pushed onto the stack when called and popped when finished.

12. LIFO Principle

- The Call Stack follows Last In First Out (LIFO).

Synchronous vs Asynchronous

13. Synchronous Code

- Synchronous code executes line by line and waits for each statement to finish.

```
console.log("A");  
console.log("B");  
console.log("C");
```

Output:

```
A  
B  
C
```

14. Asynchronous Code

- Asynchronous code starts a task and allows other code to continue executing.

```
setTimeout(() => {}, 1000);
```

Blocking vs Non-Blocking

15. Blocking Operation

- A blocking operation prevents the thread from doing any other work.

```
while(true){}
```

16. Non-Blocking Operation

- A non-blocking operation lets JavaScript continue executing other code.

```
setTimeout(() => {}, 1000);
```

17. Relationship

- Most synchronous code is blocking.
- Most asynchronous code is non-blocking.

Browser Runtime

18. JavaScript Doesn't Have Timers

- Features like `setTimeout`, `fetch`, and DOM APIs are provided by the browser.

19. Web APIs

- Browser APIs execute asynchronous tasks outside the Call Stack.

20. Callback Queue

- Completed callbacks are placed inside the Callback Queue.

21. Event Loop

- The Event Loop moves callbacks from the Queue to the Stack when the Stack becomes empty.

Promise

22. Why Promises Exist

- Promises represent values that will be available in the future.

23. Promise States

- Pending → operation running.
- Fulfilled (Resolved) → success.
- Rejected → failure.

24. Promise Meaning

- "I will give you the result later."

```
const promise = fetch("/users");
```

Async / Await

25. async Keyword

- `async` makes a function automatically return a Promise.

```
async function getUser() {  
    return "Atul";  
}
```

Internally:

```
return Promise.resolve("Atul");
```

26. await Keyword

- `await` waits for a Promise to resolve or reject.

```
const user = await getUser();
```

27. What await Actually Pauses

- `await` pauses only the current async function.

28. What await Does NOT Pause

- `await` does not pause the entire JavaScript thread.

```
async function main() {  
    const user = await getUser();  
}  
  
console.log("Still runs");
```

29. Behind the Scenes

- `await` is essentially syntactic sugar over `.then()`.

```
const user = await getUser();
```

roughly becomes:

```
getUser().then(user => {  
  // remaining code  
});
```

Browser JS vs Node.js

30. Core JavaScript is the Same

- Both Browser JS and Node.js use the same JavaScript language.

31. Both Have

- Memory Heap
- Call Stack
- Execution Context
- Event Loop
- Promises
- Async/Await

32. Browser Provides

- `window`
- `document`
- `localStorage`
- `fetch`
- `setTimeout`

33. Node.js Provides

- `fs`
- `http`
- `path`
- `crypto`
- `process`

Node.js Internals

34. Node.js Uses V8

- Node.js executes JavaScript using the V8 engine.

35. Node.js Uses libuv

- libuv provides the Event Loop and asynchronous capabilities.

36. Thread Pool

- libuv maintains a thread pool (4 threads by default).

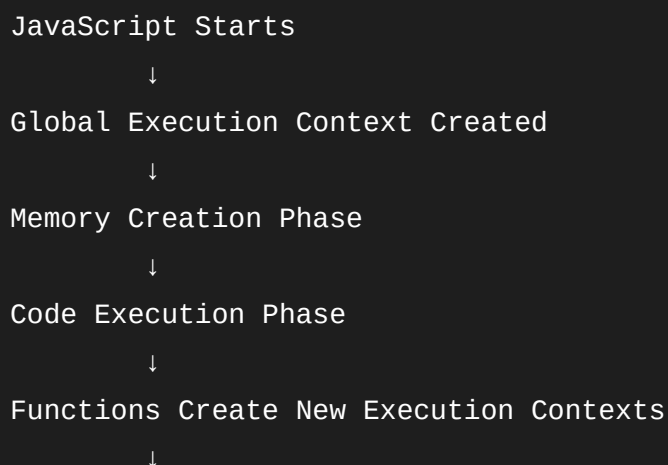
37. File System Operations

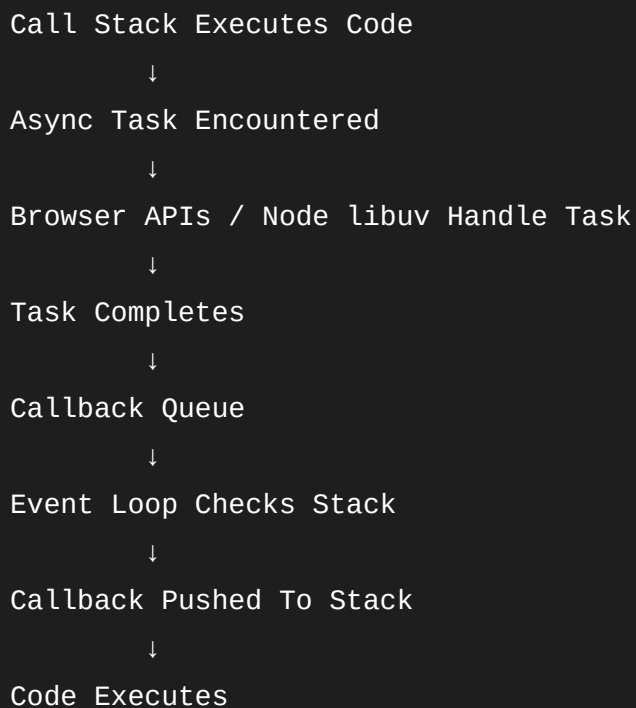
- Operations like `fs.readFile()` are handled by the thread pool.

38. Crypto Operations

- Heavy crypto work is handled by background threads.

Final Mental Model





One-Line Summary

JavaScript is a single-threaded language that executes code using a Call Stack; asynchronous operations are handled outside the stack (Browser APIs or Node's libuv), return results through Promises, and are resumed using the Event Loop, with `async/await` providing cleaner syntax for working with those Promises.

6/23/2026, 10:25:46 AM